

Informe final - FJSSP Rush Order

Scheduling dinámico de un Flexible Job Shop ante un Rush Order en un entorno Digital Twin

Curso: Gestión de Operaciones — Trabajo final **Caso:** perturbación *Rush Order* en $t^* = C_{\max 0}/2$ **Fecha:** julio 2026

1. Introducción

En manufactura personalizada, el programa de producción rara vez se ejecuta tal como se planificó: fallas de máquinas, atrasos y llegada de pedidos urgentes obligan a reprogramar en línea. El paradigma **Digital Twin (DT)** aborda este problema manteniendo una réplica digital del taller sincronizada con el estado físico, de modo que las perturbaciones se detectan al instante y disparan mecanismos de re-optimización (Liu et al., 2022).

Este trabajo aborda el **Flexible Job-Shop Scheduling Problem (FJSSP)** del caso de estudio de Liu et al. (2022) — un taller de autos personalizados con 8 trabajos, 8 máquinas y 5 operaciones por trabajo — sometido a la perturbación **Rush Order**: la llegada de un pedido urgente en el instante $t^* = C_{\max 0}/2$, donde $C_{\max 0}$ es el makespan del programa inicial.

Objetivo: diseñar e implementar una metodología que (i) resuelva el scheduling inicial, (ii) simule la llegada del rush order, (iii) recupere el programa reprogramando solo las operaciones pendientes junto al pedido urgente, y (iv) incorpore un modelo inteligente (XGBoost) que seleccione la estrategia de rescheduling de menor impacto, evaluando el desempeño con métricas cuantitativas (makespan, tiempo de término del rush, *nervousness*, tiempo computacional y porcentaje de recuperación).

2. Marco teórico

FJSSP. Extiende el job shop clásico permitiendo que cada operación se procese en un subconjunto de máquinas factibles, con tiempo dependiente de la máquina. Es NP-duro incluso en instancias pequeñas, por lo que en la práctica se resuelve con metaheurísticas (Liu et al., 2022).

Digital Twin para monitoreo y decisión. El DT mantiene el estado del taller (avance de operaciones, disponibilidad de máquinas, cola de pedidos) sincronizado con el sistema físico. Ante un evento dinámico — aquí, la inserción de una nueva orden en el sistema de pedidos — el DT lo detecta, actualiza el estado y dispara el rescheduling (Liu et al., 2022; Wang et al., 2022).

Algoritmos genéticos para FJSSP. La codificación estándar de dos segmentos MS+OS (*Machine Selection + Operation Sequence*) representa simultáneamente la asignación de máquinas y el orden de las operaciones; con operadores que preservan factibilidad (crossover POX, mutación de reasignación e intercambio) todo cromosoma decodifica a un schedule factible.

Rush order insertion. Cuando llega un pedido urgente, las operaciones ya terminadas se congelan, las que están en proceso no se interrumpen, y las pendientes se reprograman junto con las del pedido urgente (Wang et al., 2022).

Estabilidad y nervousness. Reprogramar todo desde cero maximiza el desempeño teórico pero desestabiliza el taller (cambios de asignación, re-secuenciamientos, replaneación de materiales). Moratori et al. (2010) muestran que estrategias tipo *match-up* que restringen los cambios a una ventana parcial logran calidad comparable con mucho menor perturbación; ello motiva medir la **nervousness** N del programa y usarla como criterio de decisión.

Modelo inteligente. Se usa **XGBoost** (gradient boosting de árboles) como **selector de estrategia de rescheduling**: dado el estado del taller en t^* , predice qué estrategia minimizará una función de impacto Z que combina makespan, término del rush y nervousness. Se eligió por su buen desempeño en datos tabulares pequeños, su interpretabilidad vía importancia de variables y su costo de inferencia despreciable (compatible con decisión en tiempo real).

3. Descripción del problema

Taller de ensamblaje de autos personalizados (laboratorio de Liu et al., 2022):

- **Trabajos:** 8 modelos iniciales (J1–J8); cada uno con **5 operaciones** en secuencia fija: chasis, marco, puerta izquierda, puerta derecha y cubierta frontal.
- **Máquinas:** 8 brazos robóticos (M1–M8). Cada operación tiene un subconjunto de máquinas factibles con tiempos distintos (Tablas 5 y 6 del paper; transcritas en `data/datos_base_extraidos.csv`).
- **Restricciones:** precedencia tecnológica dentro de cada trabajo; cada máquina procesa una operación a la vez; sin interrupción (*non-preemption*).
- **Métrica principal:** makespan (C_{max}).
- **Perturbación:** en $t^* = C_{max}/2$ llega un rush order J9 (reedición del modelo 8, igual que el evento dinámico reportado en el paper base) que debe integrarse al programa en curso.

4. Modelo matemático

4.1 Modelo base (FJSSP, Liu et al. 2022)

Conjuntos: trabajos $J^0 = \{1, \dots, 8\}$, máquinas $M = \{1, \dots, 8\}$, operaciones $O_j = \{1, \dots, 5\}$, máquinas factibles $A_{jh} \subseteq M$. Parámetros: tiempos p_{ijh} . Variables: asignación $x_{ijh} \in \{0, 1\}$, inicios s_{jh} , términos c_{jh} , orden en máquina y_{ijhkl} , makespan $Cmax$.

$$\min Cmax$$

sujeto a: asignación única $\sum_{i \in A_{jh}} x_{ijh} = 1$; definición de término $c_{jh} = s_{jh} + \sum_i p_{ijh} x_{ijh}$; precedencia $c_{jh} \leq s_{j, h+1}$; no solapamiento en cada máquina (par de restricciones disyuntivas con constante grande L y variable y_{ijhkl}); $Cmax \geq c_{j, h_j}$.

4.2 Extensión Rush Order

En $t^* = Cmax_0/2$ llega $J^R = \{r\}$; el conjunto pasa a $J = J^0 \cup J^R$. Del schedule inicial S^0 se conocen $s_{jh}^0, c_{jh}^0, x_{ijh}^0$. Las operaciones se clasifican en:

- **Terminadas** $F = \{(j, h): c_{jh}^0 \leq t^*\} \rightarrow$ congeladas.
- **En proceso** $I = \{(j, h): s_{jh}^0 < t^* < c_{jh}^0\} \rightarrow$ continúan sin interrupción.
- **Pendientes** $P = \{(j, h): s_{jh}^0 \geq t^*\} \rightarrow$ reprogramables.
- **Rush** $R = \{(r, h)\}$. Conjunto a reprogramar: $Q = P \cup R$.

Cada máquina queda disponible en $A_i^t = \max(t^*, \max_{(j, h) \in I: x_{ijh}^0 = 1} c_{jh}^0)$ (o t^* si no tiene operación en proceso). Las restricciones del modelo base se aplican sobre Q , agregando: $s_{jh} \geq t^*$; $s_{jh} \geq A_i^t - L(1 - x_{ijh})$; congelamiento $s_{jh} = s_{jh}^0, c_{jh} = c_{jh}^0, x_{ijh} = x_{ijh}^0 \forall (j, h) \in F \cup I$; y para la primera operación pendiente de cada trabajo, precedencia respecto del término congelado $c_{j, h-1}^0$.

4.3 Función objetivo extendida y nervousness

$$\min Z = Cmax_R + \alpha C_r + \beta N$$

donde $C_r = c_{r, h_r}$ es el término del rush order y la nervousness combina cambios de inicio y de máquina de las operaciones pendientes:

$$N = \underbrace{\gamma_1 \sum_{(j, h) \in P} |s_{jh} - s_{jh}^0|}_{N_s} + \underbrace{\gamma_2 \sum_{(j, h) \in P} (1 - x_{i^0 jh})}_{N_m}$$

El modelo se resuelve de forma metaheurística (GA); Z se usa como fitness en las variantes orientadas a urgencia/estabilidad y como criterio de etiquetado del selector inteligente.

5. Metodología

5.1 Extracción de datos

Las Tablas 5 (máquinas factibles) y 6 (tiempos) del paper base se transcribieron a `data/datos_base_extraidos.csv` ($8 \times 5 = 40$ operaciones, 81 pares operación-máquina). El rush order canónico J9 (`data/datos_rush_order.csv`) es una reedición del modelo 8, consistente con el evento “nueva orden del octavo modelo” del paper.

5.2 Scheduling inicial (GA)

Cromosoma segmentado **MS+OS**: MS asigna una máquina factible a cada operación; OS es una permutación con repetición de los trabajos (40 genes; la k -ésima aparición del trabajo j es su k -ésima operación). El **decodificador** programa cada operación en el primer hueco factible de su máquina (inserción en huecos), respetando precedencia y disponibilidad; el mismo decodificador acepta un estado inicial no vacío (ocupación de máquinas y avance de trabajos), lo que permite reutilizarlo sin cambios para el rescheduling parcial. Operadores: selección por torneo ($k=3$), crossover POX en OS y uniforme en MS, mutación por reasignación de máquina e intercambio de posiciones, elitismo. Parámetros: población 120, 300 generaciones, semilla 0.

Justificación del método: el GA es el método de referencia del propio paper base (su RLEGA es un GA mejorado), maneja de forma natural la codificación mixta asignación+secuencia del FJSSP, y su fitness es intercambiable — lo que permite usar el mismo motor para las variantes con prioridad de rush y con estabilidad.

5.3 Simulación de la perturbación

En $t^* = C_{\max}/2$ el DT lógico (capa de monitoreo implementada como regla sobre el estado del sistema de pedidos) detecta la inserción de J9, clasifica las operaciones en F/I/P, congela FUI, calcula la disponibilidad A_i^t y construye el subproblema sobre $Q = P \cup R$.

5.4 Estrategias de rescheduling

1. **insert_end** (baseline sin recuperación): el programa pendiente queda intacto; el rush se agrega al final de cada máquina.
2. **right_shift / inserción simple**: programa pendiente intacto; el rush se inserta en los primeros huecos factibles.
3. **partial_ga**: GA sobre Q minimizando $C_{\max_R}$.
4. **priority_ga**: GA sobre Q minimizando $C_{\max_R} + \alpha C_r$.
5. **stability_ga**: GA sobre Q minimizando $C_{\max_R} + \alpha C_r + \beta N$.

5.5 Selector inteligente (XGBoost)

Para cada escenario se extraen **14 features** del estado del taller en t^* (avance, cargas, holguras, tamaño y flexibilidad del rush, saturación; lista completa en `src/xgboost_selector.py`). El dataset de entrenamiento se genera sintéticamente: **300 escenarios** con rush order aleatorio en cada iteración (3–5 operaciones, 1–3 máquinas factibles, tiempos $U[30,105]$), t^* variable en $[0.3, 0.7] \cdot C_{\max}$ y cinco schedules iniciales

distintos (variación de la carga del taller). Cada escenario se resuelve con las 5 estrategias, registrando el costo Z de cada una. Distribución de estrategias ganadoras: insert_end 34, right_shift 8, partial_ga 18, priority_ga 22, stability_ga 218 (72.7%).

El desbalance de clases motivó la elección de la arquitectura. Se compararon tres formulaciones sobre el mismo split estratificado 70/30: (i) **clasificador** XGBoost de la estrategia ganadora con pesos balanceados por clase, (ii) el mismo clasificador sin balancear, y (iii) **regresor de costo** XGBoost que predice Z para cada par (estado, estrategia) y selecciona el argmin. Las dos variantes de clasificación deciden *peor* que el baseline trivial “elegir siempre stability_ga” (el balanceo agrava el problema, porque fuerza desviaciones hacia clases minoritarias que cuestan más de lo que aportan). Se adoptó la **regresión de costo**, que optimiza directamente la calidad de decisión en vez del accuracy de clasificación (validación en 6.3). partial_ga se mantiene en el repertorio: gana 18/300 escenarios, no es una clase muerta.

5.6 Diseño experimental

Cuatro escenarios obligatorios con semilla fija (reproducibles con python src/run_experiments.py): (1) scheduling inicial; (2) rush sin recuperación inteligente; (3) recuperación por GA parcial; (4) recuperación con selector XGBoost. Métricas: Cmax, Cr, tiempo computacional, N, operaciones modificadas y % de recuperación

$$= \frac{C_{max_{pert}} - C_{max_{rec}}}{C_{max_{pert}} - C_{max_0}} \times 100.$$

6. Resultados

6.1 Scheduling inicial

El GA obtiene **Cmax₀ = 382 s** en 4.9 s (Gantt en results/gantt_inicial.png). Referencias del paper base (Tabla 7, 5 corridas): RLEGA 397/400/404 (mín/prom/máx), GA 411/420/433, TS 435/453.6/466.

6.2 Perturbación y recuperación

Con t* = 191 s, el estado del taller es: 18 operaciones terminadas, 7 en proceso, 15 pendientes; Q = 15 + 5 = 20 operaciones. La cadena mínima del rush J9 suma 321 s, de modo que **ninguna solución puede terminar el rush antes de 191 + 321 = 512 s** (cota inferior).

Escenario	Estrategia	Cmax (s)	Cr (s)	N	Ops. modif.	Tiempo (s)	% Recup.
1. Inicial	GA	382	—	—	—	4.9	—
2. Rush sin recuperación	insert_end	518	518	0	0	<0.1	0
3. Recuperación GA parcial	partial_ga	512	512	747	12	0.8	4.4
4. Recuperación	XGBoost →	518	518	0	0	1.1	0

Escenario	Estrategia	Cmax (s)	Cr (s)	N	Ops. modif.	Tiempo (s)	% Recup.
inteligente	stability_ga						
(comp.)	right_shift	518	518	0	0	<0.1	0
(comp.)	priority_ga	512	512	693	13	0.9	4.4

Daño de la perturbación: $518 - 382 = 136$ s de makespan. El GA parcial alcanza **512 s, el óptimo demostrable** (coincide con la cota inferior), recuperando 6 s (4.4% del daño).
 Ganttts en results/gantt_perturbado.png, results/gantt_recuperado.png y results/gantt_recuperado_xgboost.png.

6.3 Decisión y validación del selector

Decisión en el escenario canónico. Evaluando $Z = C_{\max_R} + 0.5 \cdot C_r + 0.1 \cdot N$: insert_end/stability_ga obtienen $Z = 777$, partial_ga $Z = 842.7$ y priority_ga $Z = 837.3$. La estrategia elegida por XGBoost (**stability_ga**) es efectivamente la de menor Z : ganar 6 s de makespan costaría modificar 12 de las 15 operaciones pendientes, con 7 cambios de máquina ($N = 747$), y el modelo aprendió que ese trade-off no conviene.

Validación contra baselines (split estratificado 70/30; test $n = 90$; results/evaluacion_selector.csv):

Política	Z medio (test)	Accuracy
Oráculo (mejor estrategia por escenario)	737.9	—
Selector (regresión de Z)	740.1	0.74
Baseline trivial: siempre stability_ga	740.1	0.73
Clasificador XGBoost balanceado (descartado)	751.4	0.63

El selector **igual a al baseline trivial y supera claramente a los clasificadores**. No lo supera porque el margen total disponible es de solo 2.2 puntos de Z (740.1 vs 737.9 del oráculo, un 0.3%): en esta distribución de escenarios la política estable es casi inmejorable, y esa diferencia residual es del orden del ruido del GA que genera las etiquetas. El valor del selector no está en este margen sino en que *aprende* la política correcta en vez de asumirla — se desvía de stability solo cuando predice ventaja (2 veces en el test) — y en que la misma arquitectura re-entrena sin cambios si el taller, los pesos α/β o la distribución de rush orders cambian y la política estable deja de ser dominante. En cuanto al recall por clase (stability 1.0, insert_end 0.1, resto 0.0), confirma que con margen tan estrecho el modelo aprende a no desviarse; no es un defecto de clasificación sino la decisión de menor riesgo.

La feature más importante del regresor es el **tiempo total mínimo del rush** (0.45), seguida de su tiempo promedio (0.07) y los indicadores de estrategia — coherente con el análisis: el costo Z está gobernado por la cadena crítica del propio rush, que fija la cota inferior de C_r .

6.4 Comparación con el paper base

- **Inicial:** nuestro GA (382) mejora el mejor RLEGA reportado (397). No se afirma réplica: el decodificador con inserción en huecos construye schedules activos (el

paper no detalla el suyo), y difieren parámetros, número de corridas, semillas y entorno computacional.

- **Rescheduling:** el paper reporta $397 \rightarrow 521$ al insertar una nueva orden en $t = 200$ s; nuestro resultado $382 \rightarrow 518$ (sin recuperación) y $\rightarrow 512$ (GA parcial) es consistente en magnitud, con la misma estructura de evento ($t^* \approx 200$ s, nueva orden = octavo modelo).
- **Tiempos computacionales:** el paper no reporta tiempos del caso de estudio; los nuestros (≈ 1 s por rescheduling) son compatibles con decisión en línea.

7. Discusión

- **Calidad de solución.** El GA con decodificación por huecos es competitivo (382 vs 397 del RLEGA) y el rescheduling parcial alcanza el óptimo del subproblema (cota inferior 512), por lo que la calidad de recuperación no es mejorable con ningún otro método.
- **Impacto del rush order.** El daño (136 s) está dominado por la cadena crítica del propio rush (321 s lanzada en $t^* = 191$): el margen de recuperación vía re-secuenciamiento es estructuralmente pequeño en este escenario (máx. 6 s). Esto no es una debilidad del método sino una propiedad de la instancia: con rush orders más cortos o flexibles (como los del set de entrenamiento) las estrategias difieren mucho más.
- **Trade-off makespan–estabilidad.** El resultado central: recuperar 4.4% del makespan cuesta modificar 12 de las 15 operaciones pendientes, incluidos 7 cambios de máquina. Bajo la función Z , la mejor decisión es mantener el programa estable — exactamente lo que recomienda el selector XGBoost, en línea con el argumento de match-up de Moratori et al. (2010): no siempre conviene reoptimizar todo.
- **Rol del modelo inteligente.** stability_ga ganó en 218/300 escenarios de entrenamiento, pero en 82 ganaron otras estrategias: el repertorio no es redundante. La validación (6.3) muestra que un clasificador de la clase ganadora — balanceado o no — decide peor que el baseline trivial, mientras que la regresión de costo lo iguala; la lección metodológica es que en selección de estrategias importa la *calidad de la decisión* (Z), no el accuracy, y que un baseline fuerte debe reportarse siempre.
- **Limitaciones.** Una sola instancia base ($8 \times 8 \times 5$); pesos α , β , γ fijados por juicio; margen baseline–oráculo pequeño (2.2 en Z) y del orden del ruido de etiquetado del GA, por lo que superar al baseline trivial no es estadísticamente alcanzable en esta distribución de escenarios; nervousness medida solo sobre operaciones pendientes; detección del evento modelada como regla del DT lógico (no hay planta física); GA sin garantía de optimalidad fuera de este subproblema (aquí verificable por la cota).

8. Conclusiones

Se implementó una metodología completa de scheduling dinámico para un FJSSP en contexto Digital Twin: scheduling inicial por GA ($C_{\max_0} = 382$ s, mejor que el RLEGA reportado), simulación del rush order en $t^* = C_{\max_0}/2$ con congelamiento de operaciones terminadas/en proceso, cinco estrategias de recuperación y un selector XGBoost (regresión de costo Z) entrenado con 300 escenarios sintéticos de rush aleatorio y validado contra el baseline trivial y el oráculo. El GA parcial alcanza el makespan recuperado óptimo (512 s) y el selector inteligente identifica correctamente la estrategia de menor impacto global (Z), privilegiando la estabilidad del programa cuando la

ganancia de makespan es marginal. **El objetivo del trabajo se cumple:** el sistema detecta la perturbación, la absorbe sin interrumpir operaciones en curso y decide la recuperación con criterio cuantitativo.

Trabajo futuro: instancias múltiples y de mayor escala, calibración de α/β con el decisor, otras perturbaciones (falla de máquina, la segunda del paper base), selector entrenado sobre variantes parametrizadas continuas de estrategia, y conexión a un DT físico.

9. Referencias

- Liu, Z., Wang, Y., Liang, X., & Ma, Y. (2022). Digital twin-driven adaptive scheduling for flexible job shops. *Sustainability*, 14(9), 5340. <https://doi.org/10.3390/su14095340>
- Wang, J., Liu, Y., Ren, S., Wang, C., & Wang, W. (2022). A method for dynamic insertion order scheduling in flexible job shops based on digital twins. *Applied Sciences*, 12(23), 12430.
- Moratori, P., Petrovic, S., & Vázquez-Rodríguez, J. A. (2010). Integrating rush orders into existent schedules for a complex job shop problem. *Applied Intelligence*, 32, 205–215. <https://doi.org/10.1007/s10489-010-0215-6>

10. Anexos

A. Supuestos adoptados

1. **Pesos de la función Z:** $\alpha = 0.5$, $\beta = 0.1$, $\gamma_1 = 1$, $\gamma_2 = 10$ (un cambio de máquina ≈ 10 unidades de tiempo de perturbación). Fijados por juicio; el análisis de 6.3 muestra la decisión que inducen.
2. **Rush order canónico:** reedición del modelo 8, igual que el evento dinámico del paper base. Para el entrenamiento del selector, el rush es **aleatorio en cada iteración** (3–5 ops, 1–3 máquinas por op, tiempos $U[30,105]$ — rango empírico de la Tabla 6).
3. **Due date del rush:** no se define en el enunciado ni en el paper; se reporta C_r en su lugar y se omite la tardanza.
4. **t^* de entrenamiento:** $U(0.3, 0.7) \cdot C_{\max 0}$ para que el selector generalice alrededor del caso canónico $t^* = 0.5 \cdot C_{\max 0}$.
5. **Informe PDF:** generado con pandoc (HTML + MathML) e impresión headless de Edge, sin LaTeX.

B. Datos y parámetros

- Datos del caso: data/datos_base_extraidos.csv (Tablas 5–6 del paper); rush: data/datos_rush_order.csv.
- GA inicial: población 120, 300 generaciones, torneo $k=3$, $p_{\text{mut}} = 0.15$, elitismo 1. GA de rescheduling (experimentos): población 80, 150 generaciones. GA de entrenamiento del selector: población 30, 40 generaciones (presupuesto reducido; 300 escenarios $\times$ 5 estrategias en ≈ 60 s).
- Selector (regresor de costo): XGBRegressor, 300 árboles, profundidad 5, lr 0.08; entrada = 14 features + one-hot de estrategia (1500 filas de entrenamiento). Clasificador de comparación: XGBClassifier 200/4/0.1 con sample_weight balanceado.

C. Resultados y código

- Tabla completa: `results/tabla_resultados.csv`; importancia de features: `results/importancia_features.csv`; validación del selector: `results/evaluacion_selector.csv`.
- Código fuente en `src/` (cada módulo incluye un autochequeo ejecutable de factibilidad: precedencias, no solapamiento, congelamiento y respeto de t*).